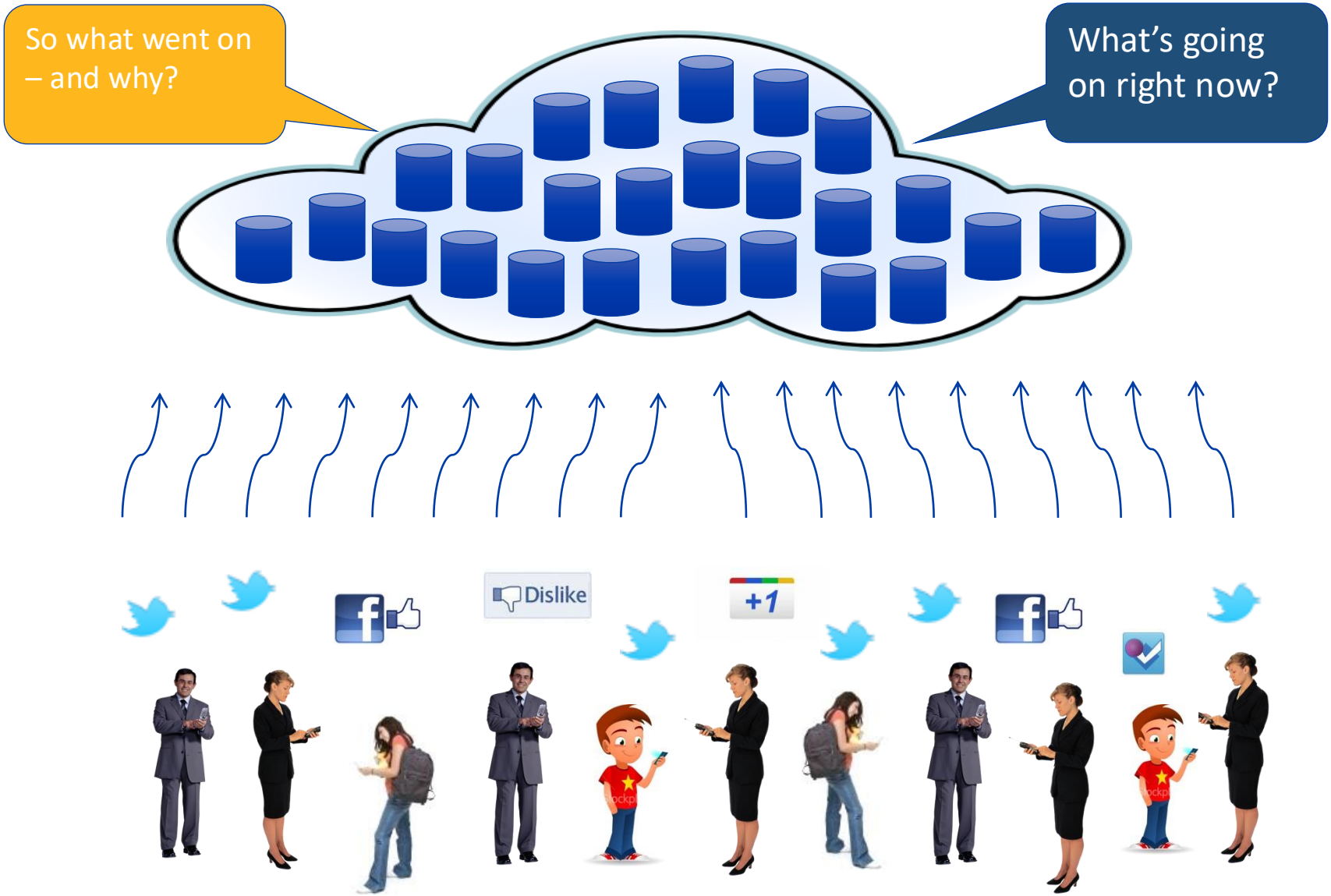
AsterixDB

A Scalable Open Source BDMS

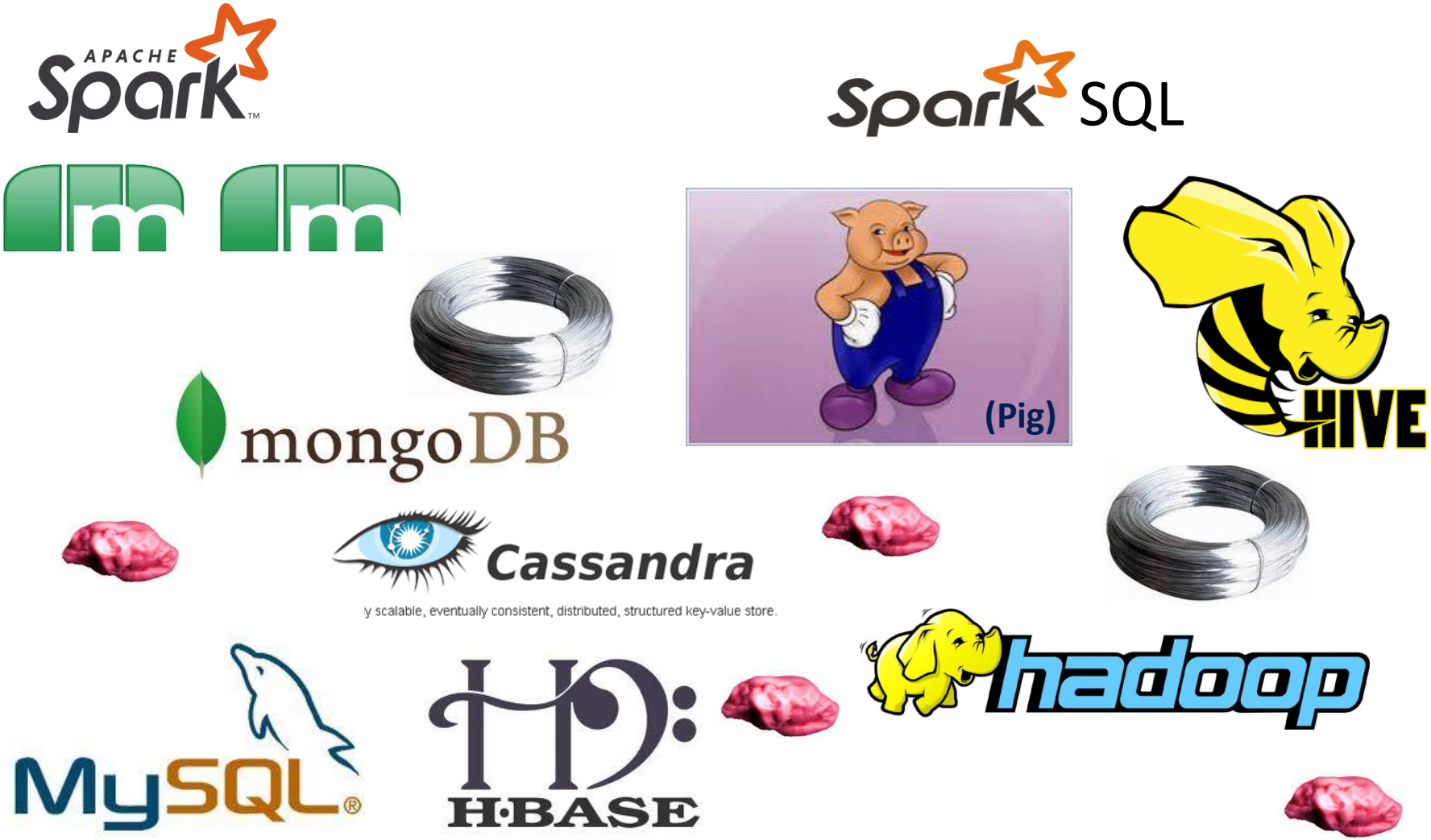
CELEBRATING 30 YEARS
Marlan and Rosemary Bourns College of Engineering

This presentation is based on slides made by Michael J. Carey, Chen Li, and Vassilis Tsotras

Big Data / Web Warehousing



Also: Today's Big Data Tangle



AsterixDB: "One Size Fits a Bunch"

Semistructured Data Management

Parallel Database Systems

1st Generation "Big Data" Systems

BDMS Desiderata:

- Able to **manage** data
- **Flexible** data model
- Full **query** capability
- Continuous data **ingestion**
- Efficient and robust **parallel** runtime
- Cost **proportional** to task at hand
- Support "**Big Data** data types"
- ⋮

ASTERIX Data Model (ADM)

```
CREATE DATAVERSE TinySocial;
USE TinySocial;

CREATE TYPE GleambookUserType AS {
   id: int,
   alias: string,
   name: string,
   userSince: datetime,
   friendIds: {{ int }},
   employment: [EmploymentType]
};
```

```
CREATE TYPE EmploymentType AS {
   organizationName: string,
   startDate: date,
   endDate: date?
};

CREATE DATASET GleambookUsers
                    (GleambookUserType)
PRIMARY KEY id;
```

Highlights include:

- JSON++ based data model
- Rich type support (spatial, temporal, …)
- Records, lists, bags
- ***Open vs. closed types***

ASTERIX Data Model (ADM)

```
CREATE DATAVERSE TinySocial;
USE TinySocial;

CREATE TYPE GleambookUserType AS {
   id: int
};
```

```
CREATE TYPE EmploymentType AS {
   organizationName: string,
   startDate: date,
   endDate: date?
};

CREATE DATASET GleambookUsers
                        (GleambookUserType)
PRIMARY KEY id;
```

Highlights include:

- JSON++ based data model
- Rich type support (spatial, temporal, …)
- Records, lists, bags
- ***Open vs. closed types***

ASTERIX Data Model (ADM)

```
CREATE DATAVERSE TinySocial;
USE TinySocial;

CREATE TYPE GleambookUserType AS {
   id: int
};

CREATE TYPE GleambookMessageType AS {
      messageId: int,
      authorId: int,
      inResponseTo: int?,
      senderLocation: geometry?,
      message: string
   };

CREATE TYPE EmploymentType AS {
   organizationName: string,
   startDate: date,
   endDate: date?
};

CREATE DATASET GleambookUsers
                        (GleambookUserType)
PRIMARY KEY id;

CREATE DATASET GleambookMessages
                        (GleambookMessageType)
PRIMARY KEY messageId;
```

Ex: GleambookUsers Data

```
{"id”:1, "alias":"Margarita", "name":"MargaritaStoddard", "nickname":"Mags”,
  "userSince":datetime("2012-08-20T10:10:00"),  "friendIds":{{2,3,6,10}},
  "employment": [ {"organizationName":"Codetechno”, "startDate":date("2006-08-06")},
                        {"organizationName":"geomedia" ,     "startDate":date("2010-06-17"),
                                                                                  "endDate":date("2010-01-26")} ],
  "gender":"F”
},

{"id":2, "alias":"Isbel”, "name":"IsbelDull", "nickname":"Izzy",
  "userSince":datetime("2011-01-22T10:10:00"),  "friendIds":{{1,4}},
  "employment": [ {"organizationName":"Hexviafind",  "startDate":date("2010-04-27")} ]
},

{"id":3, "alias":"Emory", "name":"EmoryUnk”,
  "userSince":datetime("2012-07-10T10:10:00"), "friendIds":{{1,5,8,9}},
  "employment": [ {"organizationName":"geomedia”,  "startDate":date("2010-06-17"),
                                                                  "endDate":date("2010-01-26")} ]
},
          .....
```

Other DDL Features

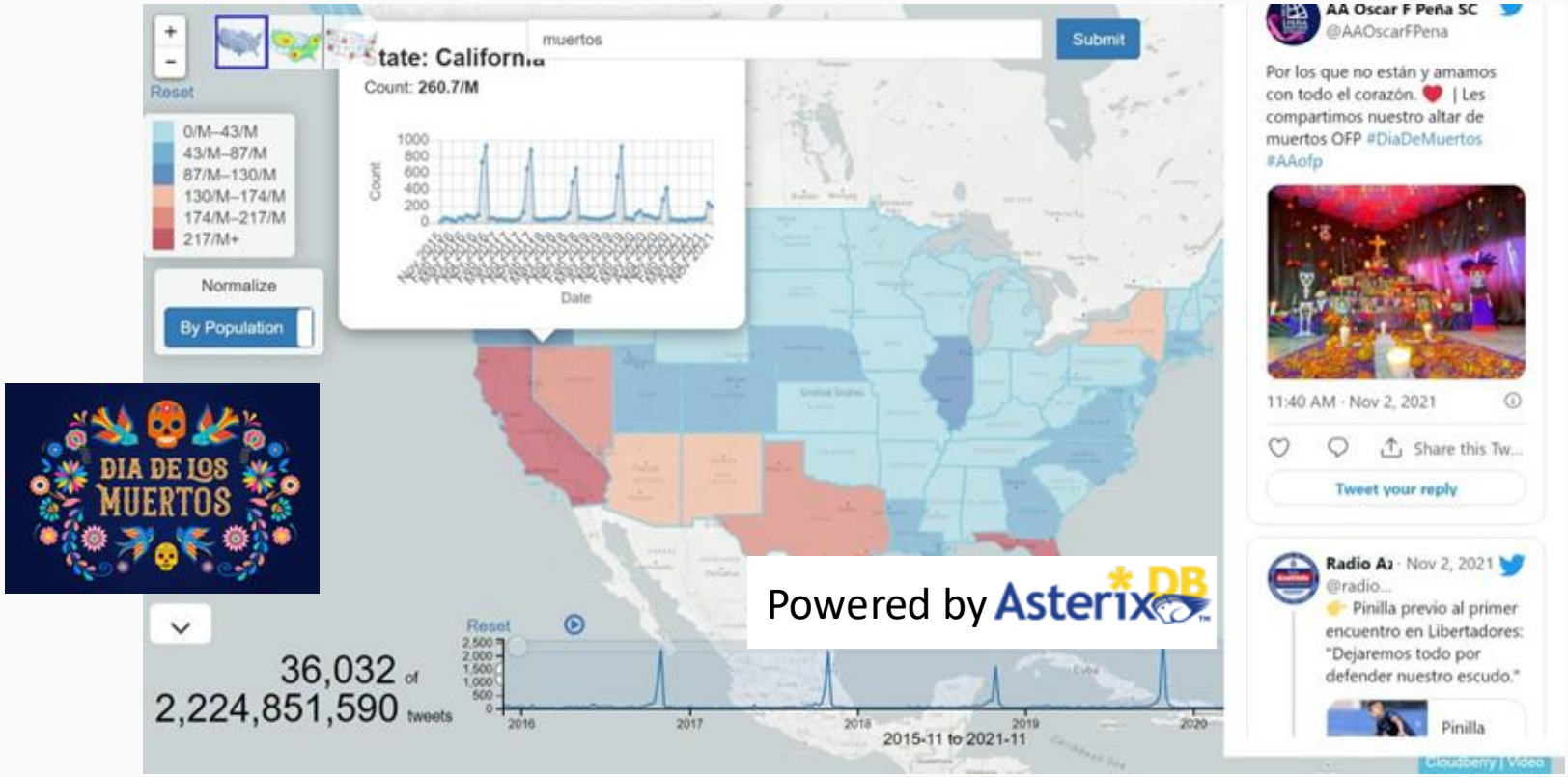
```
CREATE INDEX gbUserSinceIdx ON GleambookUsers(userSince);
CREATE INDEX gbAuthorIdx ON GleambookMessages(authorId) TYPE BTREE;
CREATE INDEX gbSenderLocIndex ON GleambookMessages(senderLocation) TYPE RTREE;
CREATE INDEX gbMessageIdx ONGleambookMessages(message) TYPE KEYWORD;
//-------------------- and also ----------------------------------------------------------------------------------
CREATE TYPE AccessLogType AS CLOSED
   { ip: string, time: string, user: string, verb: string, `path`: string,  stat: int32, size: int32 };
CREATE EXTERNAL DATASET AccessLog(AccessLogType) USING localfs
   (("path"="localhost:///Users/mikejcarey/extdemo/accesses.txt"),
     ("format"="delimited-text"), ("delimiter"="|"));

CREATE FEED myMsgFeed USING socket_adapter
    (("sockets"="127.0.0.1:10001"), ("address-type"="IP"),
     ("type-name"="GleambookMessageType"), ("format"="adm"));
CONNECT FEED myMsgFeed TO DATASET GleambookMessages;
START FEED myMsgFeed;
```

External data highlights:

- Equal opportunity access
- Feeds to “keep everything!”
- Ingestion, *not* streams

Example: Cloudberry [https://cloudberry.ics.uci.edu/]

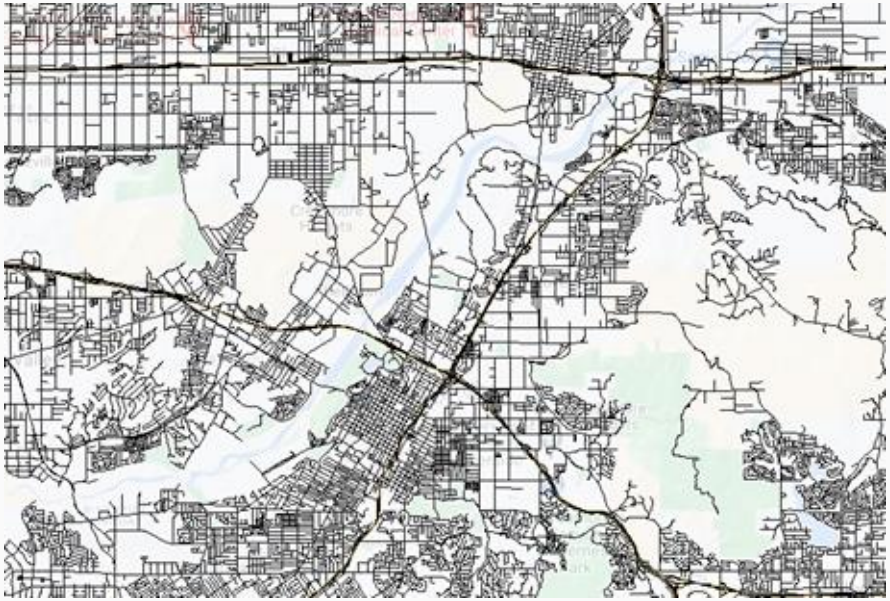


How to use AsterixDB

UCR
CELEBRATING 30 YEARS
Marlan and Rosemary Bourns
College of Engineering

TIGER 2018 - Roads


```
{
  "Roads": {
    "id": "1d07bd25-9f0d-c2ae-f211-8e7fe9d171f6",
    "g": {
      "type": "LineString",
      "coordinates": [
        [
          -97.64915899999998,
          30.153811999999988
        ],
        [
          -97.64889399999998,
          30.15415899999999
        ]
      ],
      "crs": {
        "type": "name",
        "properties": {
          "name": "EPSG:4326"
        }
      }
    },
    "LINEARID": "1103744862959",
    "FULLNAME": "Arrowmound Cove",
    "RTTYP": "M",
    "MTFCC": "S1400"
  }
}
```

Size: 9.1 GB
Number of records: 18 m
Number of points: 349 m
Geometry type: MULTILINESTRING

https://star.cs.ucr.edu/?TIGER2018/ROADS#center=33.9780,-117.3242&zoom=12

Creating Data Types and Datasets - Roads

```
USE SpatialAPI21;
CREATE TYPE RoadsType AS open {
     id:uuid,
      g:geometry
};
CREATE DATASET Roads(RoadsType) PRIMARY KEY id;

LOAD DATASET Roads USING localfs
(("path"="127.0.0.1:///data/datasets/roads.json"),
("format"="adm"));
```


```
{
 "Roads": {
  "id": "1d07bd25-9f0d-c2ae-f211-8e7fe9d171f6",
  "g": {
   "type": "LineString",
   "coordinates": [
    [
     -97.64915899999998,
     30.153811999999988
    ],
    [
     -97.64889399999998,
     30.15415899999999
    ]
   ],
   "crs": {
    "type": "name",
    "properties": {
     "name": "EPSG:4326"
    }
   }
  },
  "LINEARID": "1103744862959",
  "FULLNAME": "Arrowmound Cove",
  "RTTYP": "M",
  "MTFCC": "S1400"
 }
}
```

Yelp Business Datase

```
USE SpatialAPI21;
CREATE TYPE YelpBusinessType AS OPEN {
      id:uuid,
      name:string,
      latitude:double,
      longitude:double
};

CREATE DATASET YelpBusiness(YelpBusinessType) PRIMARY
KEY id;

LOAD DATASET YelpBusiness USING localfs
(("path"="127.0.0.1:///data/datasets/yelp_business.json
"),
("format"="adm"));
```


```
{
 "YelpBusiness": {
   "id": "781cfac6-8edd-4da4-41e1-64d843fd0948",
   "name": "Lane Wells Jewelry Repair",
   "latitude": 30.346169,
   "longitude": -97.711458,
   "business_id": "N3_Gs3DnX4k9SgpwJxdEfw",
   "address": "7801 N Lamar Blvd, Ste A140",
   "city": "Austin",
   "state": "TX",
   "postal_code": "78752",
   "stars": 5,
   "review_count": 30,
   "is_open": 1,
   "attributes": {
    "BusinessAcceptsCreditCards": "True","DogsAllowed":
"True","RestaurantsDelivery": "None","BusinessAcceptsBitcoin":
"False","BikeParking": "True","RestaurantsTakeOut":
"None","WheelchairAccessible": "True"
   },
   "categories": "Shopping, Jewelry Repair, Appraisal Services, Local
Services, Jewelry, Engraving, Gold Buyers",
   "hours": {
    "Monday": "12:15-17:0","Tuesday": "12:15-17:0","Wednesday": "12:15-
17:0","Thursday": "12:15-17:0","Friday": "12:15-17:0"
   }
 }
}
```

Yelp Reviews

```
USE SpatialAPI21;
CREATE TYPE YelpReviewType AS OPEN {
    id:uuid,
    text:string,
    review_date:date?
};

CREATE DATASET YelpReview(YelpReviewType) PRIMARY KEY id;

LOAD DATASET YelpReview USING localfs
(("path"="127.0.0.1:///data/datasets/yelp_reviews.json"),
("format"="adm"));
```


```
{
 "YelpReview": {
  "id": "e29e9020-40b1-ef38-39e2-0a81f508101a",
  "text": "I think their rice dishes are way better than their Pho's. Also I really like their sandwiches and spring rolls. I took a friend who haven't had viennese food ever. He absolutely love it.",
  "review_id": "GDgXjXSZCA1iNQWD7OHXfg",
  "user_id": "1RCRKuHgP3FskGUVnmFdxg",
  "business_id": "mOnesB4IF9j6-ZmHoOHOig",
  "stars": 4,
  "useful": 0,
  "funny": 0,
  "cool": 0,
  "date": "2017-05-26 03:05:46"
 }
}
```

External Dataset

```
// External datasets
CREATE TYPE RoadsTypeExternal IF NOT EXISTS AS {
  g: geometry,
  FULLNAME: String
};

CREATE EXTERNAL DATASET RoadsExternal(RoadsTypeExternal)
USING localfs (("path"="127.0.0.1:///data/datasets/roads.json"),
("format"="adm"));

SELECT VALUE r
FROM RoadsExternal r
LIMIT 1;
```

- Good for in-situ processing of static data (Similar to Hadoop and Spark).
- Files living in HDFS or in the local filesystem(s) of a cluster's nodes are currently supported.

Modification Statements

- Delete

```
USE SpatialAPI21;
DELETE FROM YelpReview WHERE id = uuid("e29e9020-40b1-ef38-39e2-0a81f508101a");
```

-

```
USE SpatialAPI21;
UPSERT INTO YelpBusiness (
  SELECT st_make_point(b.longitude, b.latitude) AS g, b.*
  FROM YelpBusiness b
);
```

Query 1

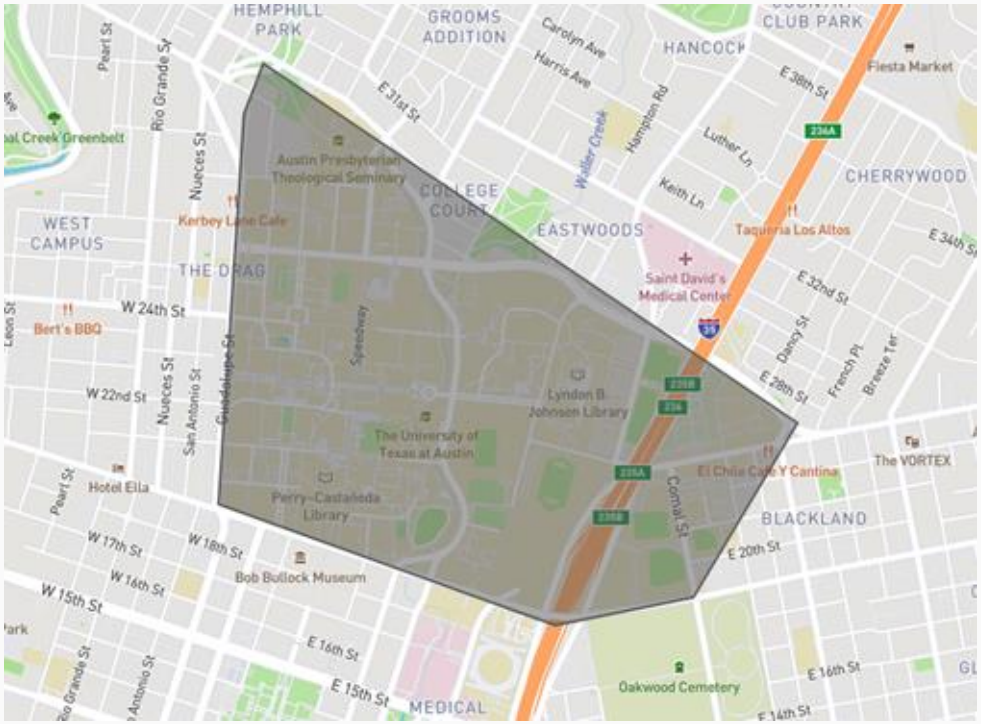
Create a new geometry type field from doubles longitude and latitude.

```
USE SpatialAPI21;
UPSERT INTO YelpBusiness (
  SELECT st_make_point(b.longitude, b.latitude) AS
g, b.*
  FROM YelpBusiness b
);
```


```
{
  "YelpBusiness": {
    "id": "781cfac6-8edd-4da4-41e1-64d843fd0948",
    "name": "Lane Wells Jewelry Repair",
    "latitude": 30.346169,
    "longitude": -97.711458,
    "business_id": "N3_Gs3DnX4k9SgpwJxdEfw",
    "address": "7801 N Lamar Blvd, Ste A140",
    "city": "Austin",
    "state": "TX",
    "postal_code": "78752",
    "stars": 5,
    "review_count": 30,
    "is_open": 1,
    "attributes": {
      "BusinessAcceptsCreditCards": "True","DogsAllowed":
"True","RestaurantsDelivery": "None","BusinessAcceptsBitcoin":
"False","BikeParking": "True","RestaurantsTakeOut":
"None","WheelchairAccessible": "True"
    },
    "categories": "Shopping, Jewelry Repair, Appraisal Services, Local
Services, Jewelry, Engraving, Gold Buyers",
    "hours": {
      "Monday": "12:15-17:0","Tuesday": "12:15-17:0","Wednesday": "12:15-
17:0","Thursday": "12:15-17:0","Friday": "12:15-17:0"
    }
  }
}
```

Query 2

Find the businesses in the University of Texas at Austin.

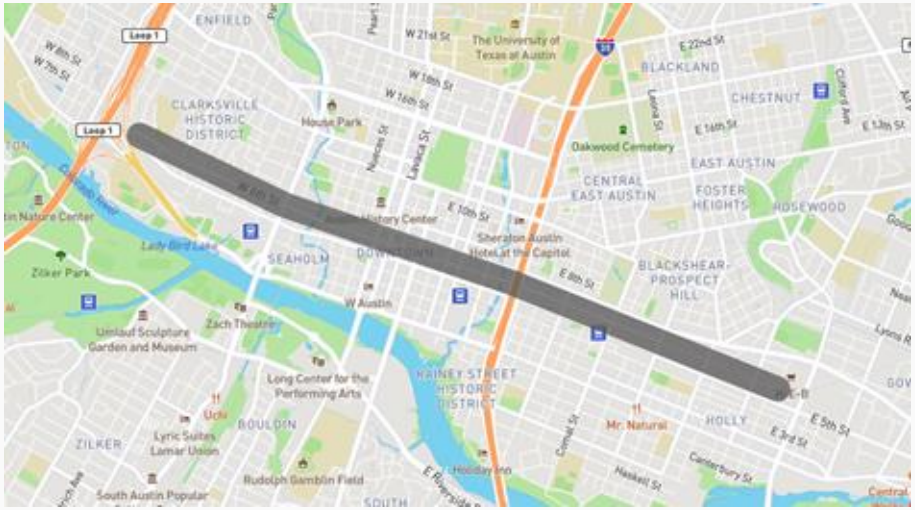



```
USE SpatialAPI21;
SELECT * FROM YelpBusiness where
st_contains(st_geom_from_geojson({
        "type": "Polygon",
        "coordinates": [
          [
            [-97.7411127090454,30.293868239039856],
            [-97.74201393127441,30.28193572785586],
            [97.73008346557617,30.278229683682895],
            [-
97.72519111633301,30.279082086236702],
            [-97.72150039672852,30.28441869914471],
            [-97.7404260635376,30.295424546529237],
            [-97.7411127090454,30.293868239039856]
          ]
        ]
      }), g);
```

Query 3

Find the average stars for each category of the businesses that are located nearby the 6th Street in Austin, TX.

```
USE SpatialAPI21;
SELECT
    c, AVG(b.stars) AS average_stars
FROM
    YelpBusiness b, word_tokens(b.categories) c
WHERE
    st_distance(
        b.g,
        st_geom_from_text('LINESTRING (-97.76613235473631
30.276598979906108, -97.7539873123169 30.271521387805628, -
97.73510456085205 30.265961679168175, -97.71278858184814
30.258288764226883)')
    ) < 0.001
GROUP BY
    c
ORDER BY
    average_stars;
```



Query 4

Finding the number of five-star reviews for each road.

```
USE SpatialAPI21;
select
      r.FULLNAME, count(1) c
from
      RoadsAustin r, YelpAustin b, YelpReviewAustin a
where
      st_distance(r.g, b.g) < 0.0003 and
      a.business_id = b.business_id and
      a.stars = 5
group by
      r.LINEARID, r.FULLNAME
order by
      c desc;
```


```
[
    {
        "c": 4469,
        "FULLNAME": "E 6th St"
    },
    {
        "c": 3447,
        "FULLNAME": "Congress Ave S"
    },
    {
        "c": 3152,
        "FULLNAME": "S 1st St"
    },
    {
        "c": 2885,
        "FULLNAME": "S Lamar Blvd"
    },
    {
        "c": 2885,
        "FULLNAME": "State Loop 343"
    },
    {
        "c": 2757,
        "FULLNAME": "E Cesar Chavez St"
    },
    {
        "c": 2502,
        "FULLNAME": "Guadalupe St"
    },
    {
        "c": 2172,
        "FULLNAME": "W 6th St"
    },
    {
        "c": 2104,
        "FULLNAME": "Burnet Rd"
    },
    {
        "c": 1997,
        "FULLNAME": "Red River St"
    }
]
```

Query 5

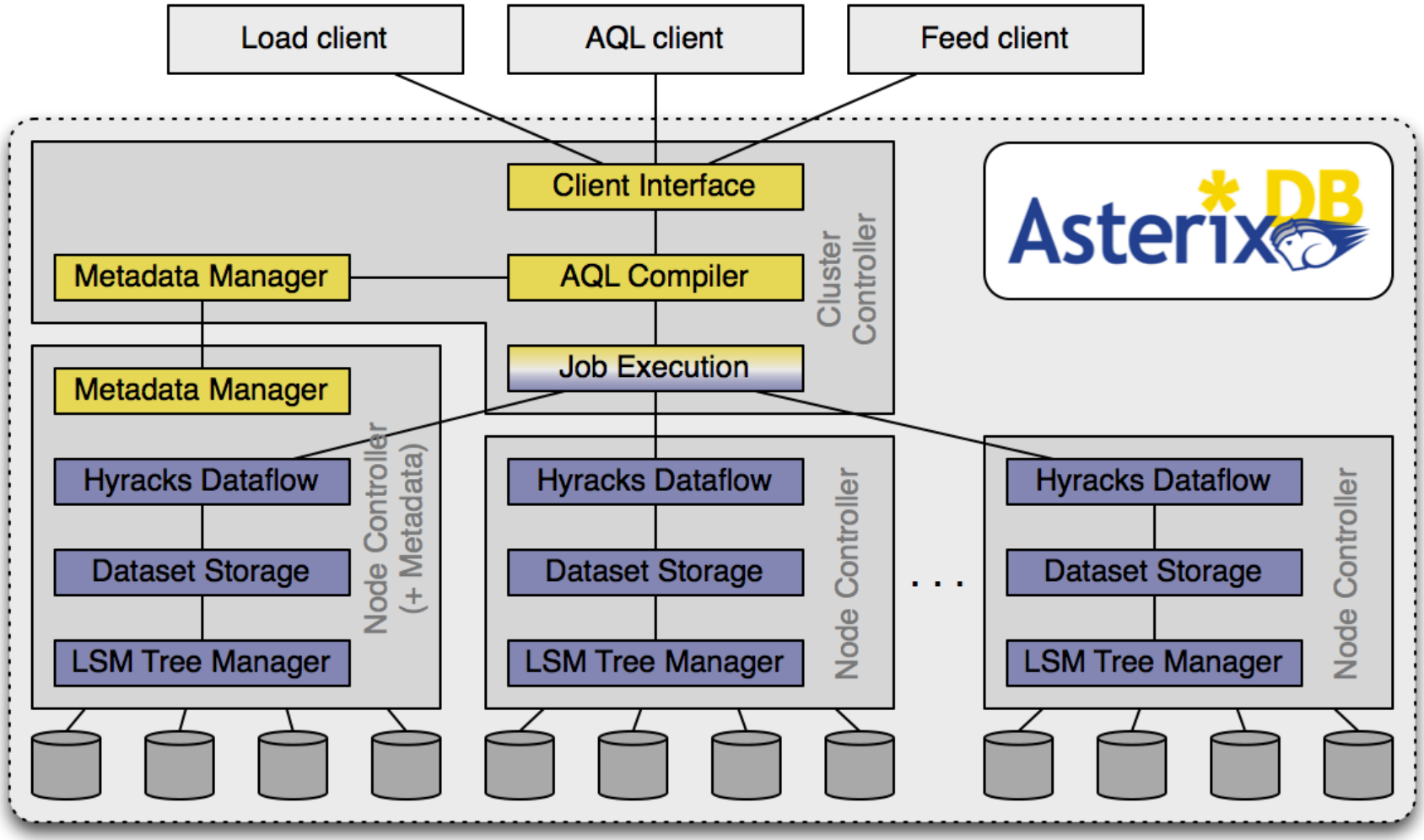
Finding the top ten roads that have the most five-stars businesses relative to their length.

```
USE SpatialAPI21;
SELECT
    "FeatureCollection" as `type`,
    {"name":r.FULLNAME} as properties,
    (select "Feature" as `type`, {} as properties, brg.y.g as geometry
from BRGroup as brg) as features
FROM
    YelpBusiness y,
    Roads r
WHERE
    st_distance(r.g, y.g) < 0.0003 and
    y.stars = 5
GROUP BY
    r.LINEARID, r.FULLNAME, r.g
GROUP AS
    BRGroup
ORDER BY (count(1)/st_length(r.g)) DESC
LIMIT 10;
```

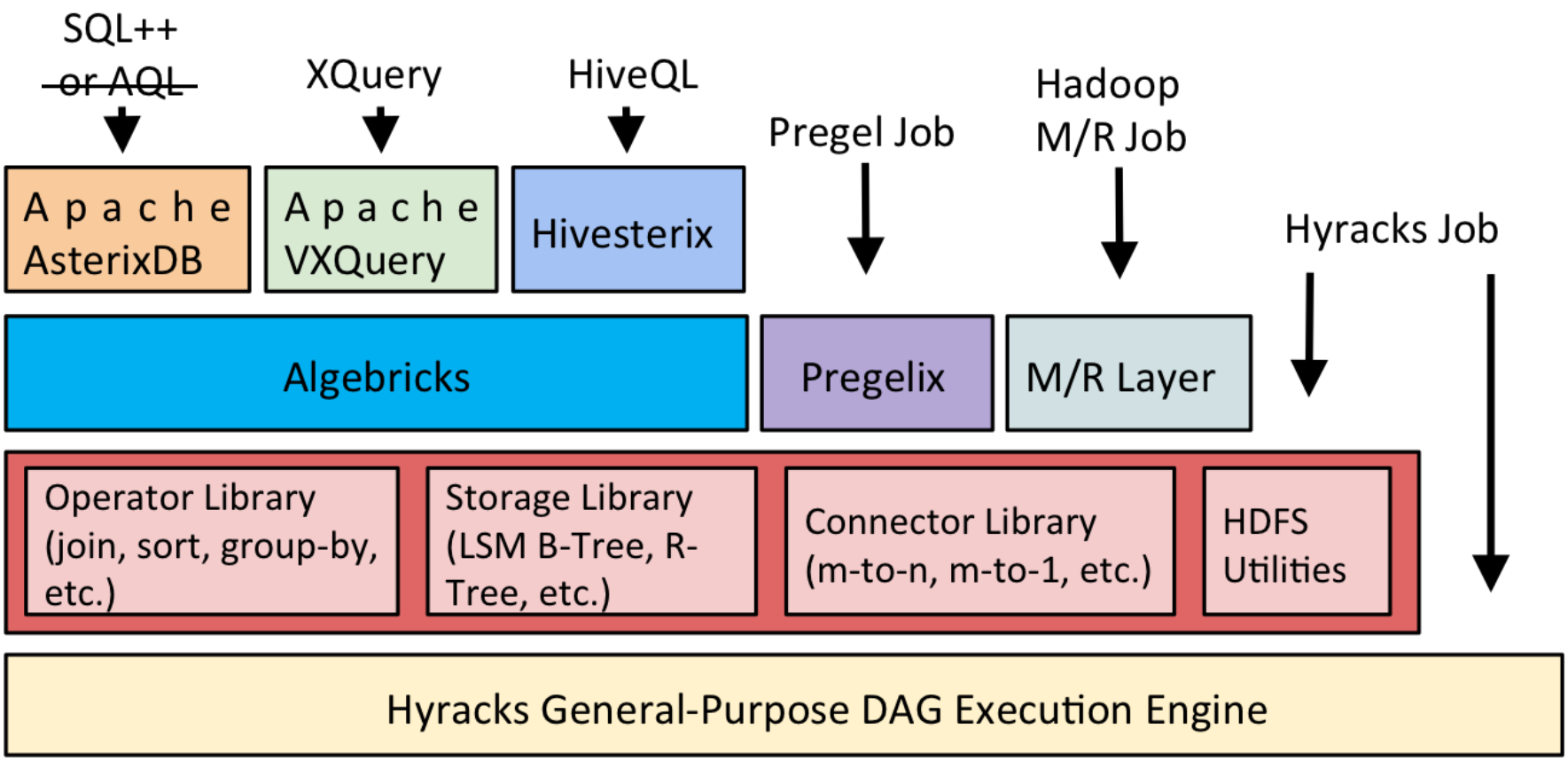
AsterixDB Architecture and Design

UCR
CELEBRATING 30 YEARS
Marlan and Rosemary Bourns
College of Engineering

AsterixDB System Overview



Software Stack



Hyracks Dataflow Runtime

- Partitioned-parallel platform for data-intensive computing
- Job = dataflow DAG of operators and connectors
  - Operators consume and produce ***partitions*** of data
  - Connectors ***route*** (repartition) data between operators
- Hyracks *vs.* the "competition"
  - Based on time-tested parallel database principles
  - *vs.* Hadoop MR: More flexible model and less "pessimistic"
  - *vs.* newer SQL-on-Hadoop runtimes: Emphasis on out-of-core execution and adherence to memory budgets
  - Fast job activation, data pipelining, binary format, state-of-the-art DB style operators (hash-based, indexed, ...)
- Early test at Yahoo! Labs on 180 nodes (1440 cores, 720 disks)

Hyracks *(cont.)*

Query

```
use dataverse TinySocial

avg (
 for $m in dataset MugshotMessages
 where $m.timestamp >=
     datetime("2014-01-01T00:00:00")
  and $m.timestamp <
     datetime("2014-04-01T00:00:00")
 return string-length($m.message)
)}
```

Select Options | Clear Query | Run

- [ ] Print parsed expressions
- [ ] Print rewritten expressions
- [ ] Print logical plan
- [ ] Print optimized logical plan
- [x] Print Hyracks job
- [x] Execute query

Algebricks

Partitioned Parallelism!

```
aggregate $agg := global-avg($lagg)
```

n:1 replicating

```
aggregate $lagg := local-avg($l)
```

1:1

```
assign $l := string-length($m.message)
```

1:1

```
select $t >= 2014-01-01T00:00:00 and
       $t <  2014-04-01T00:00:00
```

1:1

```
assign $t := $m.timestamp
```

1:1

```
btree $m := search(MugshotMessages, $id, $id)
```

1:1

```
sort $id
```

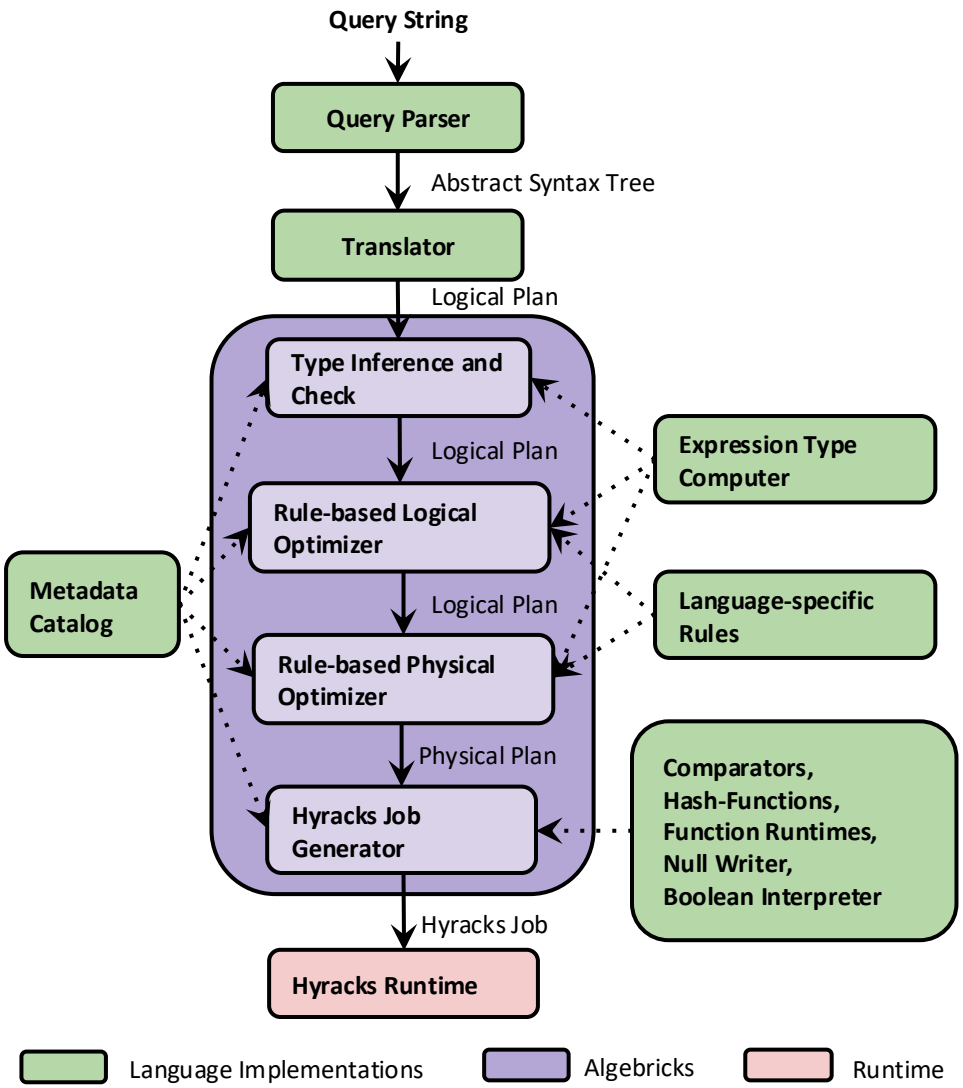
1:1

```
btree $id := search(msTimestampIdx, $lo, $hi)
```

1:1

```
assign $hi := 2014-04-01T00:00:00
assign $lo := 2014-01-01T00:00:00
```

Algebricks Query Compiler Framework



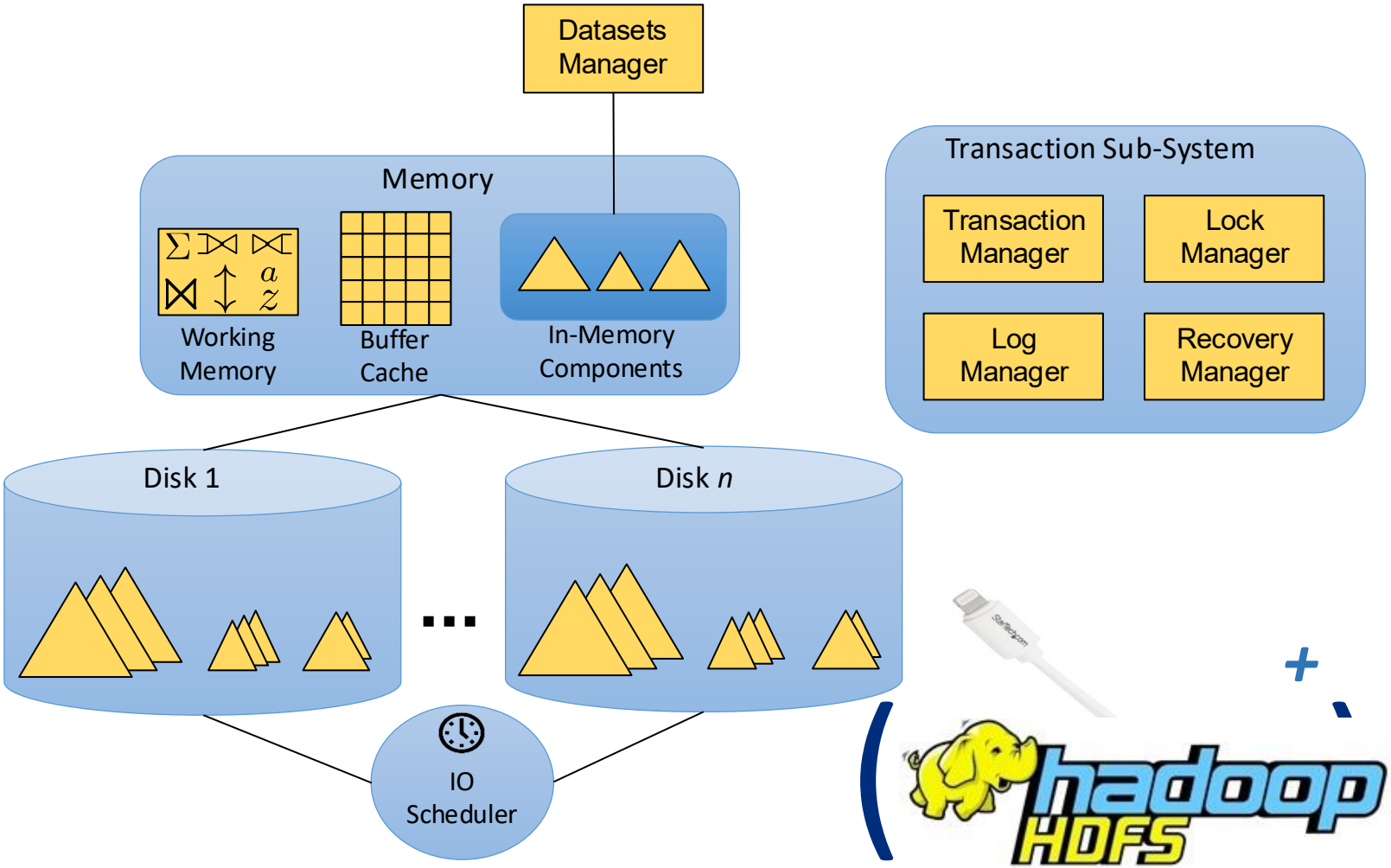
Algebricks

- Logical Operators
- Logical Expressions
- Metadata Interface
- Model-Neutral Logical Rewrite Rules
- Physical Operators
- Model-Neutral Physical Rewrite Rules
- Hyracks Job Generator

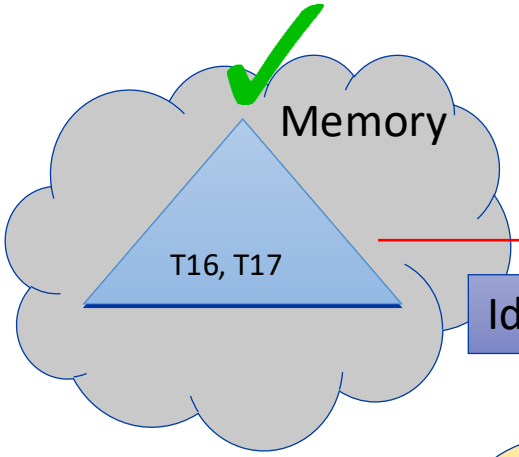
Target Query Language

- Query Parser (AST)
- AST Translator
- Metadata Catalog
- Expression Type Computer
- Logical Rewrite Rules
- Physical Rewrite Rules
- Language Specifics

Native Storage Management



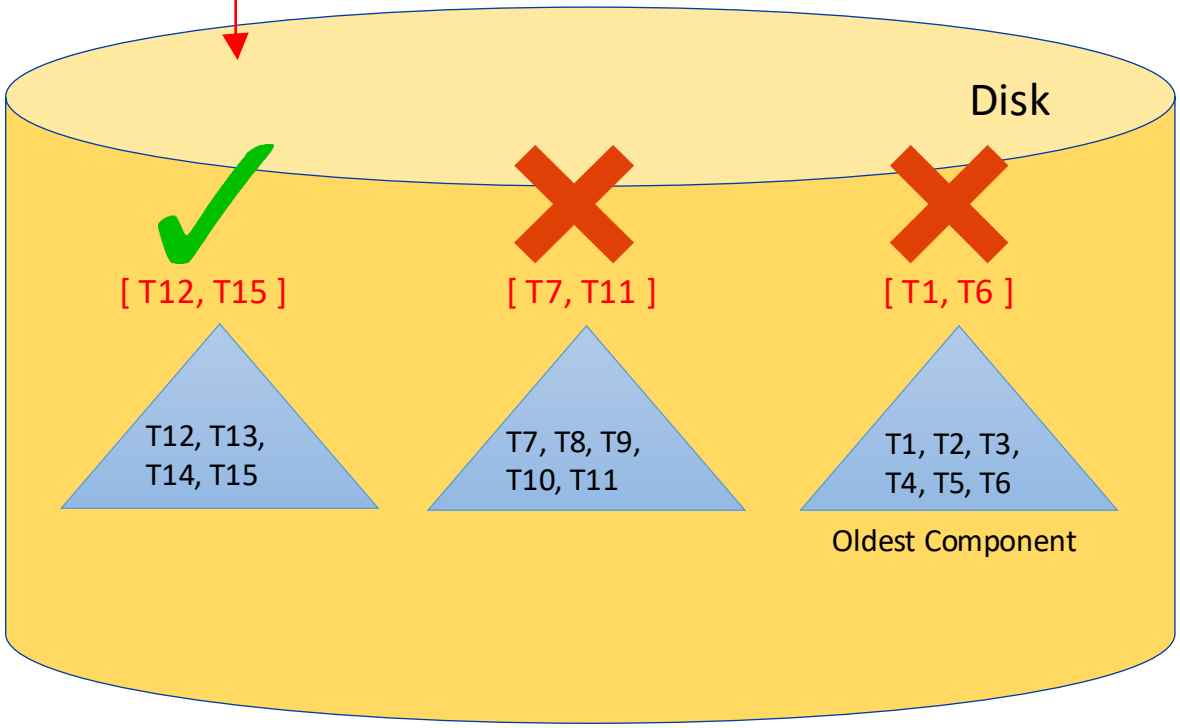
LSM-Based Filters



Intuition: Do NOT touch unneeded records

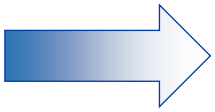
Idea: Utilize LSM partitioning to prune disk components

Q: Get all tweets > T14



Transaction Support

- Key-value store-like transaction semantics
  - Entity-level transactions (by key) within "transactors"
  - Atomic insert, delete, and upsert (including indexing)
  - Concurrency control (based on entity-level locking)
  - Crash recovery (based on no-steal logging + shadowing)
  - Backup and restore support (just in case... ☺)
- Expected use of AsterixDB is to model, capture, and ***track*** the "state of the world" (not to ***be*** it)...



SELECT ... FROM Weather W...
// return current conditions by city

(Long serializable reads)

Example AsterixDB Use Cases

- Potential use case areas include
  - Behavioral science
  - Cell phone event analytics
  - Social data analytics
  - Public health
  - Cluster management log analytics
  - Power usage monitoring
  - IoT data storage and querying
  - ....

For More Information

- Asterix project UCI/UCR research home
  - http://asterix.ics.uci.edu/
- Apache AsterixDB home
  - http://asterixdb.apache.org/
- SQL++ Primer
  - http://asterixdb.apache.org/docs/0.9.4.1/sqlpp/primer-sqlpp.html